當架構設計被 AI 加速後，

工程判斷力的養成與傳承該怎麼辦？

Jed Lin 林承毅　ClearForge 躍準科技

今天的五段

01 一個消失中的養成路徑

02 AI 讓我們欠了什麼債

03 篩選器：單向門與雙向門

04 把取捨還原成流程

05 三個訊號與一個警告



SPEAKER

林承毅 Jed Lin

ClearForge 躍準科技｜創辦人

Facebook
歡迎掃碼交流

現在 幫技術團隊導入 Agentic Coding
交付系統陪跑

經歷 20+ 年軟體開發與技術管理
線上遊戲・HR 系統・企業軟體

講者 社群分享
twMVC・DevOpsDays Taiwan・Taipei Agile Community

過去的架構設計為什麼慢

過去：一個重要的設計決策

2–3 天

避免過度設計、也避免設計不足；套一個架構風格之前，先確認它合不合當下的 context。

Vibe coding 之後：跟 AI 對話

2 小時

就有一份看起來很完整的架構文件。



> *programming properly should be regarded as an activity by which the programmers form or achieve a certain kind of insight, a theory, of the matters at hand.*

— **Peter Naur** 圖靈獎得主 ·〈Programming as Theory Building〉，1985

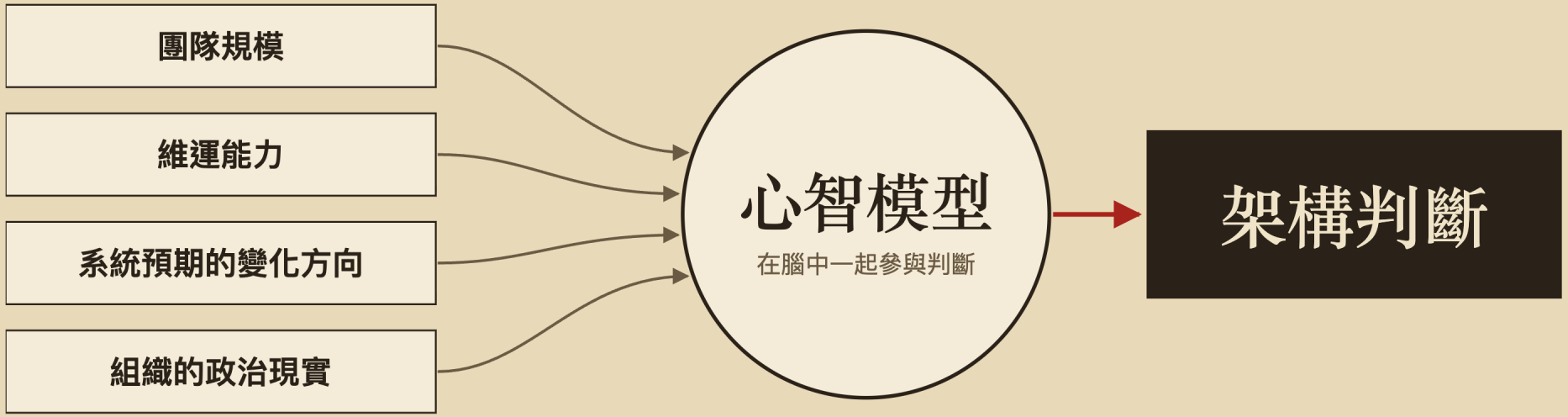
講白話

程式的核心不是那些程式碼，而是開發者腦中建立的心智模型。

慢的過程裡藏著兩件事

01 context 的內化

02 方案 Trade-off



不是在解題，是在建構解決問題的模型

慢的過程裡藏著兩件事

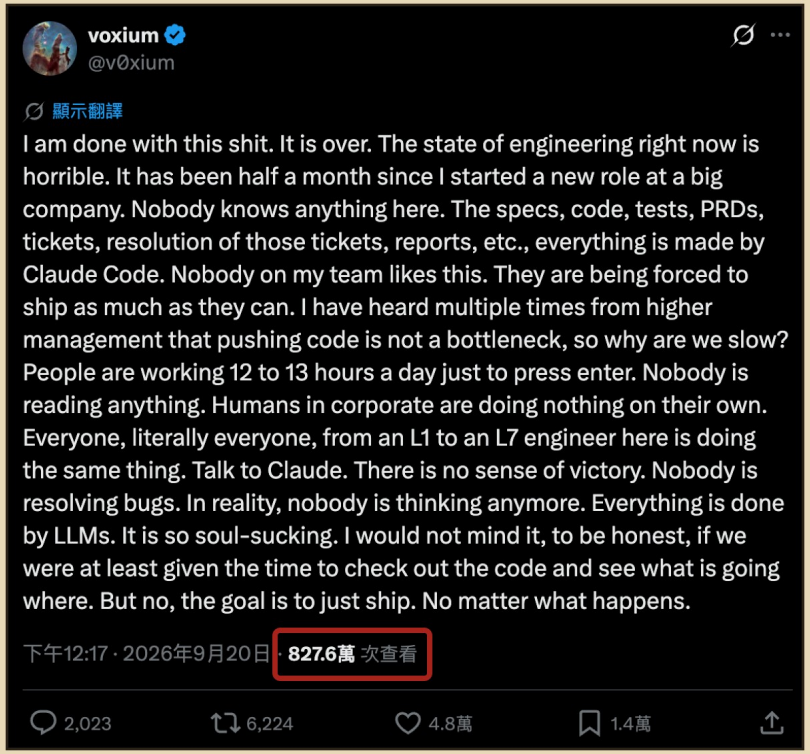
01 context 的內化

02 方案 Trade-off



判斷力，就在這個不舒服的過程裡長出來

現場的症狀：沒有人真的懂這套系統



在 X 超過 800 萬次瀏覽的一則貼文

全部是 AI 產的
一天十三小時按 Enter
最資淺到最資深都一樣

@v0xium · X · 2026 年 9 月

原始出處：https://x.com/v0xium/status/2010526107128529120

兩個斷層

96%

開發者不完全信任 AI 寫出來的程式碼

Sonar〈2026 State of Code Developer Survey〉

不到一半

每次都會檢查

下一代

失去養成的路徑

沒有老本可吃，也沒有人懂這套系統可以教

資深者

失去交接的場合

老本會吃完；設計討論被壓縮，傳承的場合也一起不見

AI 壓縮了產出時間，犧牲的是理解深度

如果這兩件事也被一起壓縮掉

01 ~~context 的內化~~　　02 ~~方案 Trade-off~~　　→ 品質流失

更嚴重的後果｜研究：認知債 Cognitive Debt

技術債活在程式碼裡，
認知債活在工程師的腦中。

— Margaret-Anne Storey



AI 還了一種債，欠下另外兩種

三債模型出自 Giorgio Polvara〈The Persistence of Theory〉(2026 年 6 月)

活在程式碼裡

技術債

重構、清理、補測試——AI 很強，而且只會更強。

AI 能還

活在人的腦中

認知債

團隊成員沒有建立解決問題的心智模型

欠下 → 下一代長不出判斷力

活在口耳相傳裡

意圖債

領域知識靠一代傳一代；人走bug 還在。

欠下 → 資深者交接不出去

案例一｜預約 SaaS 的價目表

自動解析價目表

店家給的價目表：大致有結構的自然語言

Ivy 總監｜資歷 10 年｜剪髮 1200 / 染燙 3500 起

洗剪800 女生加200 學生9折 小安

→ **系統要的是乾淨的結構化 JSON**

丟給 AI 設計

第一輪	**prompt 裡舉例，教它怎麼解析** 遺漏、甚至幻覺—測不過
第二輪	**改用 regex 字串比對** 很精準，測試全過—活了下來
第三輪	**？** 稍後回來講

認知債

解析邏輯為什麼要這樣設計，我沒有真的想過—AI 提、我接受。

意圖債

「格式會不斷變形、維護成本不能隨客戶數成長」只活在我腦中。

先自己設計，再用AI找缺漏



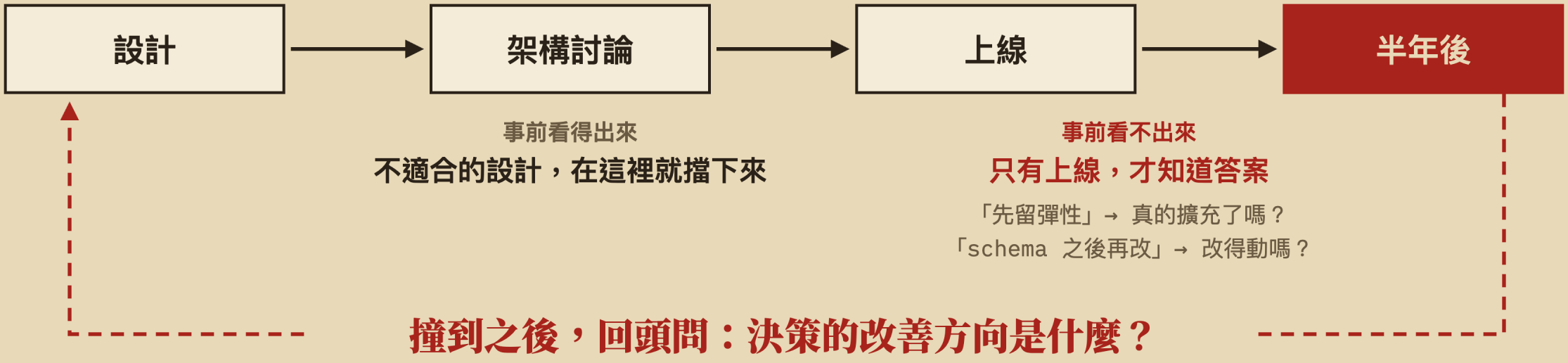
AI 先設計，本身就是問題——
因為它講得太完整。

生成模式	一張白紙	→	自己生出設計	→ ✓ 心智模型建起來
評價模式	AI 的設計文件	→	按「同意」	→ ✗ 沒建起來

生成效應（generation effect）：自己生成答案建立的理解，遠優於閱讀現成的答案。

還債的四件事 ① 先自己設計 ② 被現實檢驗 ③ 說得出取捨 ④ 拿真實情境來練

架構設計必須被現實檢驗



必須說得出取捨

AI 給的理由

決策：預留多租戶抽象層
理由
1. 擴充性：未來支援多租戶時，無需大幅重構
2. 可維護性：關注點分離，降低模組耦合
3. 業界實踐：符合 SaaS 架構的最佳實踐

取捨
- 初期開發成本略增，但長期效益顯著
- 增加一定複雜度，可透過完善文件緩解

替代方案：單租戶架構—實作簡單，但未來擴充成本高

連取捨都寫好了，而且放到哪個團隊都對。

說得出的取捨

「不做多租戶抽象。
半年內用不到，
真的要做時長什麼樣也說不準——
現在做只是在猜。」

有我們的處境，也講了放棄什麼。

拿掉 AI 的文件，還講得出來嗎？

還債的四件事 ① 先自己設計 ② 被現實檢驗 **③ 說得出取捨** ④ 拿真實情境來練

必須拿真實情境來練

練習題

沒有 context，
也沒有做錯的代價

意圖債欠的，就是這些 context

使用者是誰
團隊會什麼
系統往哪裡長

如果每個決策都要花時間想清楚，AI 帶來的好處就全部吐回去了。

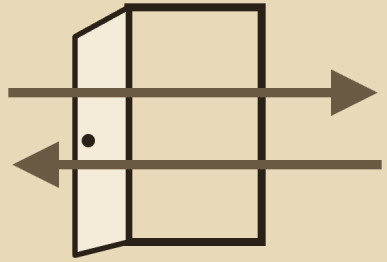
哪些決策值得花時間想，哪些可以放給 AI 全速跑？

單向門與雙向門

TWO-WAY DOOR

雙向門

錯了，退回來就好

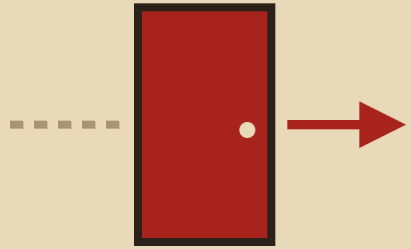


▶ 快速決策，放手讓 AI 跑

ONE-WAY DOOR

單向門

錯了，回頭 ≈ 重做



▶ 慢下來，集中判斷力

怎麼判定？四個問題

看的不是重不重要，是回頭的代價

← 雙向門　　單向門 →

01 影響範圍有多大？	一個模組內部	外部客戶
02 之後會有多少東西依賴它？	沒人依賴	生產資料、外部 API
03 錯誤多快會被發現？	下禮拜	半年後
04 要改回來，需要誰點頭？	自己開 PR	跨團隊、客戶、停機

沒有AI的舊時代，典型清單

← 錯了，退回來就好

錯了，回頭 ≈ 重做 →

典型的雙向門

- 命名
- 函式內部的實作（介面不變）
- 工具型 library
- feature flag 包起來的功能

灰色地帶

本質可逆，但重寫太貴 → 只好當單向門

- 模組內部結構與設計模式
- 只有自己在呼叫的內部介面
- 一個服務的第一版實作
- prototype

典型的單向門

- Bounded Context 邊界與資料所有權
- 對外 API 契約
- 核心技術選型
- 多租戶隔離模型
- 計費資料模型

很多門，不是被依賴關上的，是被重寫成本關上的。

AI 時代，門的位置開始有些變化

← 錯了，退回來就好

錯了，回頭 ≈ 重做 →

雙向門 ← 變成雙向門 **單向門**

重寫的成本，被 AI 壓到接近零 → 真的變成雙向門

- 模組內部結構與設計模式
- 只有自己在呼叫的內部介面
- 一個服務的第一版實作
- prototype

但單向門貴的，從來不在程式碼 → 仍是單向門

資料
三個月的生產資料與交易紀錄

約定
別人的系統已經串上的 API

習慣
使用者已經養成的操作

信任
使用者對產品的第一印象

單向門變少了，但變純了。

AI 讓程式碼變成雙向門，也讓程式碼以外的門，關得更快。

AI 時代，最容易犯的兩種誤判

誤判一｜把單向門當成雙向門

「API 先發布第一版，之後再快速迭代新版本」

看起來　新版本，AI 幾分鐘就生得出來

其實　有人串上來，第一版就成了永久的契約

誤判二｜把雙向門當成單向門

「每個內部設計開會」

看起來　慎重、不出錯

其實　內部設計早是雙向門，單向門反而分不到心力

兩種誤判，都是問錯了問題

不是「現在能不能改」，是「長滿依賴之後還能不能改」



不是戒 AI，是改順序

AI 照用、速度照拿，只調整人跟 AI 出手的先後；全部融入既有儀式，不新增會議。

1 順序紀律

2 各自先寫

3 決策重訪

4 影子決策

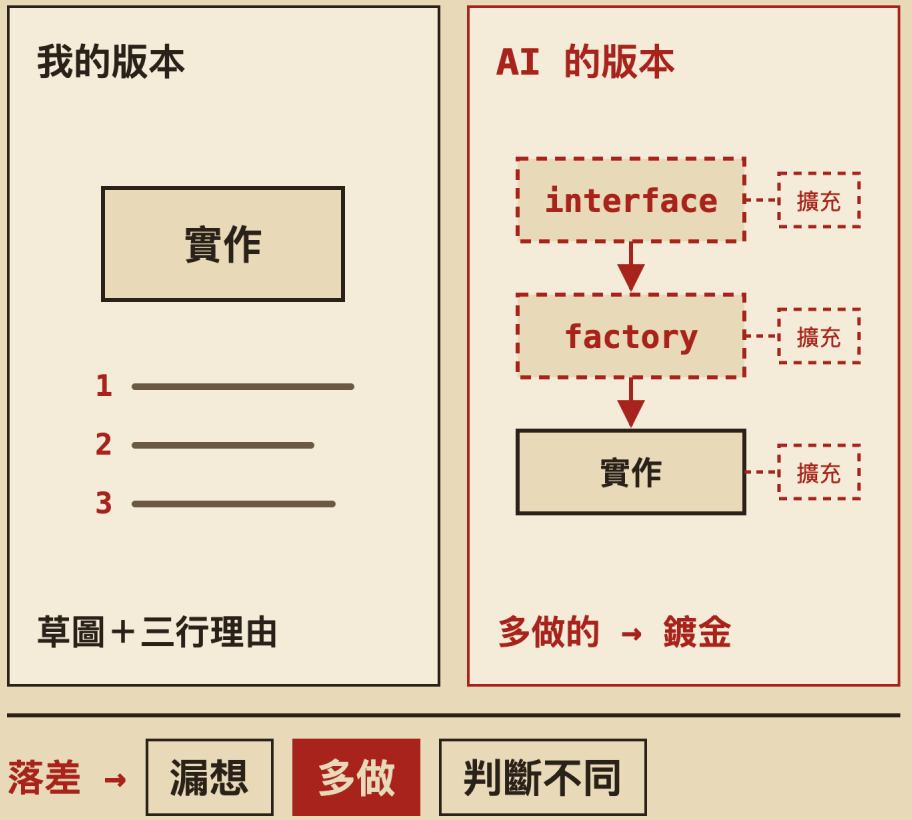
順序紀律：人先設計，AI 找缺漏

1. **碰到單向門，先花 15–30 分鐘自己寫**
   哪怕只是一張草圖，加上三行理由

2. **才讓 AI 出方案**
   要它給兩三個方案並各自陳述 `trade-off`，不收第一提案

3. **兩份放在一起比**
   我漏想了什麼？它多做了什麼？我們為什麼判斷不同？

對照｜一比就知道



案例一（續）

價目表那兩筆債，後來怎麼算的

第一輪

prompt 裡舉例教它解析

遺漏、幻覺 → 測不過

第二輪

regex 字串比對

測試全過。但每接一個新客戶就重寫一輪規則，設定檔越長越多。

第三輪：架構重寫

先讓模型理解：價目表為什麼長這樣

店家寫價目表，是為了讓客人做出消費決定。理解這件事，抽取就變得簡單。

如果第一天先寫下這兩行約束

格式會不斷變形
維護成本不能隨客戶數成長

→ regex 當場就會被否決

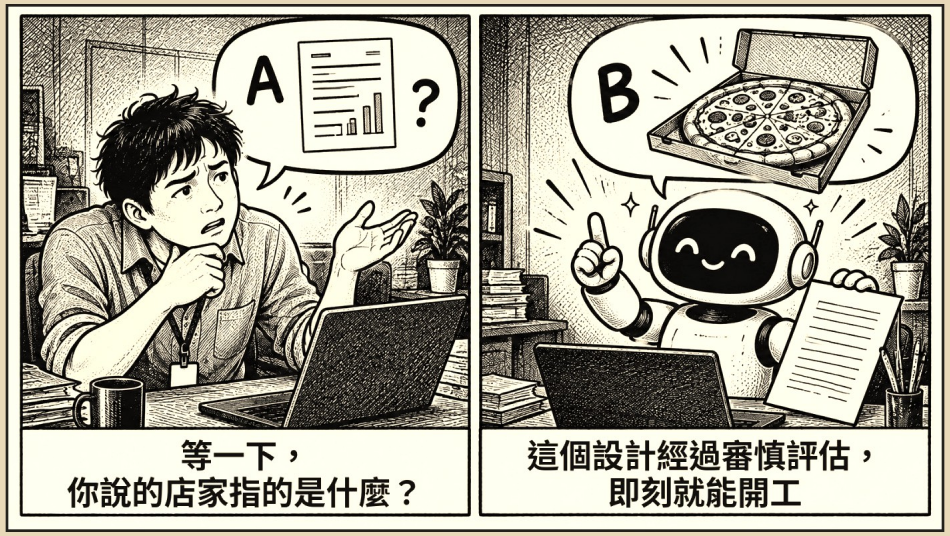
各自先寫

碰到單向門 → 甲的粗稿 | 乙的粗稿 | 丙的粗稿 各自獨立、半小時 → **攤開比對**

規則

團隊成員必須同時陳述一個被自己否決的替代方案，以及否決的理由。

「我也考慮過事件驅動，但我們沒有人維運過訊息佇列，而且這個模組的吞吐量根本用不到，所以我否決了。」



我自己的反面教材：登入機制

AI 的建議

Google Authenticator 六位數動態密碼

我否決傳統密碼的理由

「比較安全」「不用記密碼」

——剛好就是 AI 給我的那兩句

使用者測試

裝 app、掃 QR code、每 30 秒換一次

對店家是天書 → 緊急改回傳統密碼：是雙向門救了我，不是我的判斷



評審快篩

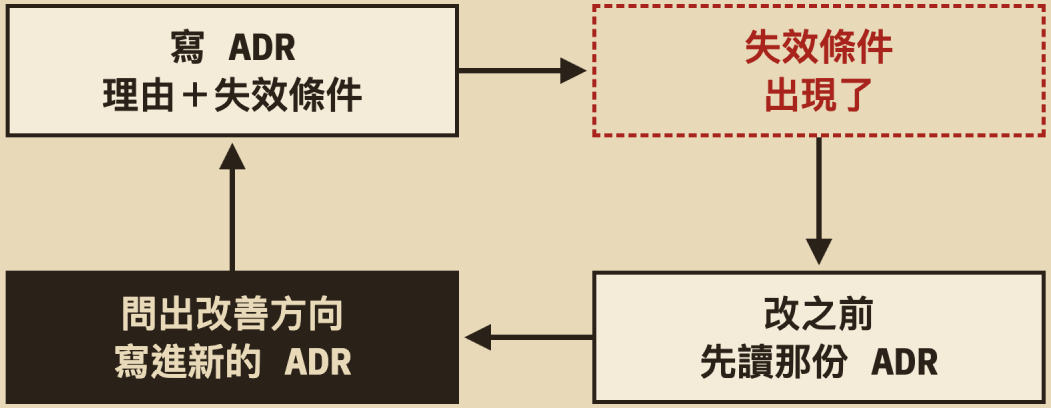
否決的理由裡，有沒有出現「我們的使用者」「我們的團隊」？

另外兩個機制：決策重訪、影子決策

機制三｜還認知債裡最難發現的那塊：模型建錯了

決策重訪

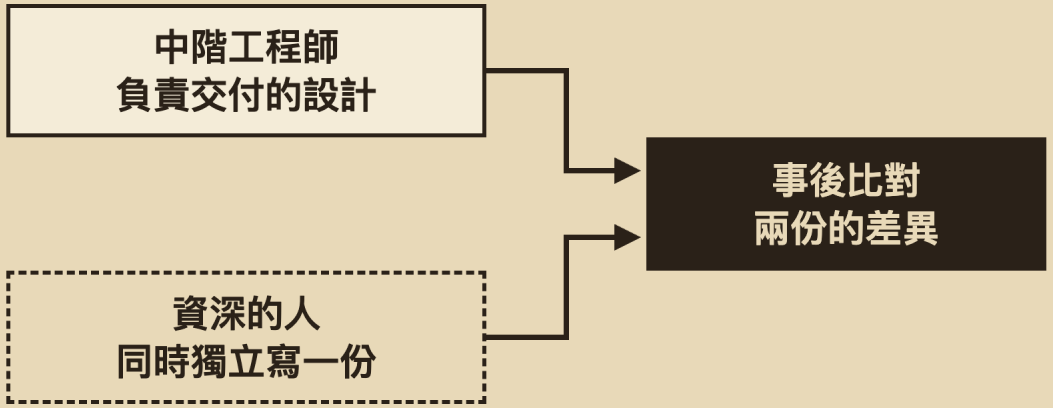
不需要新工具，就是 ADR—前提是真的在寫：



機制四｜給有一點規模的組織：師徒機制

影子決策

情境百分之百真實，也不佔用額外的決策時間：



→ 資深的版本不一定比較好，雙方都在學

決策科學稱為 ShadowBox 訓練法，原本用於消防與軍事領域的專家判斷傳承

不同規模，一頁對照

規模	怎麼開始
一個人	把 AI 當陪練；ADR 照寫，並寫進失效的情況；靠公開寫作與社群分享，逼自己把取捨說出來
新創小團隊	全部塞進既有儀式：單向門決策各自先寫，提案時說出被否決的方案與理由；改舊決策之前先讀一次當初的紀錄，不多開會
中型以上	再加上影子決策；新人 onboarding 改成「架構考古」：重建一塊 legacy 當年為什麼會那樣設計

一個大原則

判斷力，過去內化在自己身上；現在要變成可以被引用的文字，和可以被執行的流程。

還認知債

順序紀律、決策重訪——讓心智模型真的長在人的腦子裡，長錯時也有機會被發現。

還意圖債

把 context 寫進 CLAUDE.md、架構原則文件——寫明刻意不做什麼、為什麼。

這些文字，既是給 AI 的護欄，也是給下一代工程師的教材。

怎麼知道它有效：三個訊號

01

說得出改善方向

回頭看舊決策時，從「當初就是這樣寫的」，變成「漏掉了使用者是店家，下次先確認使用者是誰」。

02

能說明系統如何運作與為什麼要這樣設計

這塊架構為什麼長這樣？除了能清楚說明系統怎麼運行，也要能說明當初為什麼不採用其他方案。

03

最重要的是不要問A答B

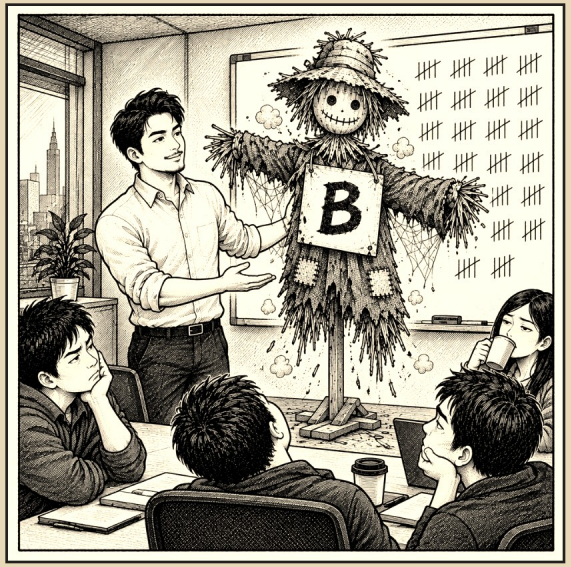
B 方案是真的想過

提案時說出的否決方案，從湊數用的，變成真的比較過的選項。

這些機制最大的敵人：形式化

當這些味道出現的時候

練習開始用假案例
ADR 每一份的失效條件都寫得差不多
「各自先寫」變成開會前先互相通氣
B 方案，永遠是同一個湊數的



寧可砍掉頻率，保住真實性。

一次真的把 context 補進文件的重訪，勝過十次走過場。

問題

AI 拿走的，是判斷力的養成路徑

篩選

只有單向門，值得慢下來

機制

順序紀律．各自先寫

決策重訪．影子決策

把被壓縮掉的取捨，還原成流程。

這可能是 AI 時代，技術主管最值得做的一筆投資。

我是 Jed，謝謝大家。

Jed Lin 林承毅 · ClearForge 躍準科技

Q&A